# Experiment # 13
# Understanding of Inverted Residual Block & Bottleneck

## Objective/s:

- What is inverted residual block and bottleneck block
- Implementation of inverted residual block and bottleneck
- Lab Tasks

## Theory:

An inverted residual block is a key component of MobileNetV2, a neural network architecture designed for mobile and resource-constrained environments. The inverted residual block is an evolution of the traditional residual block, optimized for computational efficiency and performance in lightweight models.

**Key Characteristics**

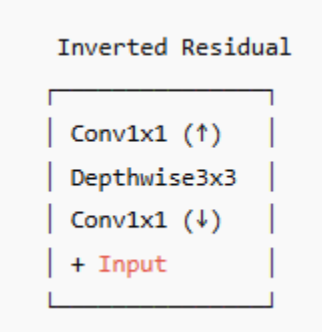
1. **Inverted Bottleneck:** Unlike traditional residual blocks that start with a wide layer and narrow down, inverted residual blocks start with a narrow layer (bottleneck) and expand to a wider layer.
2. **Depthwise Separable Convolutions:** These convolutions split the convolution operation into two parts: depthwise convolution (applies a single filter per input channel) and pointwise convolution (1x1 convolution that combines the outputs of the depthwise convolution).
3. **Linear Bottleneck:** After expanding, the block uses a linear layer without a non-linearity (ReLU) at the end, which helps preserve information and avoids losing important features.

## Structure

An inverted residual block consists of the following key components:

1. **Pointwise Convolution (1x1)**: Expands the number of channels.
2. **Depthwise Convolution (3x3)**: Applies a single convolutional filter per input channel, maintaining the number of channels but processing spatial information.

3. **Pointwise Convolution (1x1)**: Projects the expanded channels back to a lower dimension.
4. **Activation Function (ReLU)**: Applied after each batch normalization except the last one.
5. **Residual Connection**: The input is added to the output if the input and output dimensions' match.




**Bottleneck:**

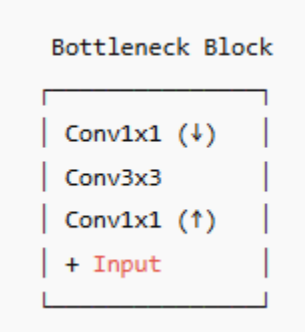
In the context of deep neural networks, particularly residual networks (ResNets) and their variants, a **bottleneck block** is a type of residual block designed to reduce computational complexity and memory usage while maintaining or improving model performance. Bottleneck blocks achieve this by using a three-layer architecture that first reduces the dimensionality of the input (compression), processes the compressed representation, and then expands the dimensionality back to the original size (decompression).

**Structure of a Bottleneck Block**

A typical bottleneck block consists of the following components:

1. **Pointwise Convolution (1x1) for Compression**: Reduces the number of channels.
2. **Convolution 2D (3x3)**: Processes the compressed representation with a convolution that maintains the number of channels.
3. **Pointwise Convolution (1x1) for Expansion**: Expands the number of channels back to the original size.
4. **Batch Normalization**: Applied after each convolution to normalize the activations.
5. **Non-linearity (ReLU6):** Activation function applied after each convolutional layer.

6. **Shortcut Connection**: The original input is added to the output of the block, enabling the residual learning.




## Lab Task:

You have to make a network in which you have 2 residual block, 2 inverted residuals and 3 bottleneck block. And you get the parameter <10 in it.